

## B. Step 2 — Persistence PostgreSQL & JPA/Hibernate

Fiche de révision EILMS

### B.1 Dépendances Maven

À retenir : postgresql (driver JDBC) + hibernate-core (implémentation JPA) + HikariCP (pool de connexions réutilisables, évite de rouvrir une connexion TCP à chaque requête).

### B.2 JDBC bas niveau

À retenir : DriverManager.getConnection(url, user, password) négocie une session brute avec la base. PreparedStatement (pas Statement) protège contre les injections SQL.

```
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM warehouse_items WHERE item_id = ?");
ps.setString(1, id);
```

### B.3 Annotations JPA de base

À retenir : @Entity + @Table mappent une classe à une table ; @Id = clé primaire ; @Column personnalise chaque colonne. Le constructeur sans argument est OBLIGATOIRE (réflexion Hibernate).

```
@Entity
@Table(name = "warehouse_items")
public class WarehouseItemEntity {
    @Id
    @Column(name = "item_id", length = 50)
    private String id;
    @Column(name = "item_name", nullable = false)
    private String name;
    public WarehouseItemEntity() {} // requis par JPA
}
```

### B.4 Relations @OneToMany / @ManyToOne

À retenir : le côté @ManyToOne porte la vraie clé étrangère (@JoinColumn) ; le côté @OneToMany utilise mappedBy et ne crée AUCUNE colonne en base.

```
// Côté Warehouse (1)
@OneToMany(mappedBy = "warehouse", cascade = CascadeType.ALL)
private List<WarehouseItemEntity> items;

// Côté Item (N) – propriétaire de la relation
@ManyToOne
@JoinColumn(name = "warehouse_id")
private WarehouseEntity warehouse;
```

Erreur fréquente : mettre @JoinColumn des deux côtés → colonnes en double ou table de jointure inutile.

### B.5 persistence.xml

À retenir : hibernate.dialect (SQL spécifique PostgreSQL) + hibernate.hbm2ddl.auto=update (crée/modifie les tables automatiquement au démarrage — pratique en dev, risqué en prod).

### B.6 EntityManagerFactory vs EntityManager

**EntityManagerFactory** : lourd, thread-safe, créé UNE SEULE FOIS par application.

**EntityManager** : léger, NON thread-safe, créé par transaction/thread — ne jamais le partager entre threads.

## B.7 Transactions ACID

**À retenir :** toute écriture entre `begin()` et `commit()` ; `rollback()` en cas d'erreur pour garantir l'atomicité.

```
try {
    tx.begin();
    em.persist(item);
    tx.commit();
} catch (RuntimeException e) {
    if (tx.isActive()) tx.rollback();
    throw e;
} finally {
    em.close();
}
```